

MODULE 3

Préparation et visualisation des données

Plan du module

01

Nettoyage

Doublons, types, formats

02

Valeurs manquantes

Détecter, supprimer, imputer

03

Données aberrantes

IQR, z-score, boxplot

04

Encodage

Label / One-Hot Encoding

05

Normalisation

Min-Max, standardisation

06

Analyse exploratoire

describe(), corr(), groupby

07

Visualisation

Matplotlib / Plotly

08

TP final

Préparation complète d'un dataset

Un modèle n'est jamais meilleur que ses données

- Des doublons faussent les statistiques et les moyennes.
- Des valeurs manquantes font planter ou biaisent un modèle.
- Des valeurs aberrantes tirent les résultats vers l'extrême.
- Des données mal préparées donnent un modèle peu fiable, quelle que soit sa sophistication.

Nettoyage

- `df.duplicated()` repère les lignes dupliquées
- `df.drop_duplicates()` supprime les doublons
- `df.dtypes` / `.astype()` vérifie et corrige les types
- `df.rename(columns=)` renomme des colonnes mal nommées
- `.str.strip()` / `.str.lower()` uniformise le texte

COMMENT ÇA MARCHE

1. Repérer et supprimer les doublons

On compare chaque ligne à toutes les précédentes pour repérer celles strictement identiques, puis on ne garde que la première occurrence. Fonctions utilisées : `duplicated()` combinée à `sum()`, puis `drop_duplicates()`.

2. Corriger un type de colonne

Quand une colonne censée être numérique est stockée comme du texte (souvent à cause de valeurs invalides mélangées), on la reconvertit dans le bon type pour pouvoir calculer dessus. Fonction utilisée : `astype()`.

3. Uniformiser une colonne texte

Deux valeurs identiques pour un humain (espaces en trop, majuscules) sont vues comme différentes par l'ordinateur. On nettoie donc le texte pour que les regroupements fonctionnent correctement. Fonctions utilisées : `str.strip()` et `str.lower()`.

Un seul dataset, du début à la fin

```
import pandas as pd
import numpy as np
url = "https://raw.githubusercontent.com/shaadyalii/Dirty-Cafe-Sales-Dataset/main/dirty_cafe_sales.csv"
df = pd.read_csv(url)

print("Nombre de lignes :", len(df))
print(df.dtypes)
df.head()
```



EXERCICE

Nettoyer un dataset brut

1. Comptez puis supprimez les doublons avec duplicated() et drop_duplicates().

```
nb_doublons = df.duplicated().sum()
df = df.drop_duplicates()
```

2. Vérifiez les types avec dtypes et corrigez 'Quantity' en entier.

```
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].median()).astype(int)
```

3. Uniformisez 'Location' : espaces en trop + minuscules.

```
df['Location'] = df['Location'].astype(str).str.strip().str.lower()
```

4. Affichez le DataFrame nettoyé et comparez les lignes avant/après.

```
print(len(df)) # avant vs après nettoyage
```

Valeurs manquantes

- `df.isna().sum()` compte les valeurs manquantes par colonne
- `df.dropna()` supprime les lignes concernées
- `df.fillna(valeur)` remplace par une constante
- `fillna(df[c].mean())` impute par la moyenne / médiane

COMMENT ÇA MARCHE

1. Repérer les trous

On parcourt le tableau colonne par colonne et on compte, pour chacune, le nombre de cases vides. Fonction utilisée : `isna()` combinée à `sum()`.

2. Supprimer les lignes incomplètes

Quand une information manquante ne peut pas être devinée, on retire complètement la ligne concernée du tableau. Fonction utilisée : `dropna()`.

3. Remplacer par une valeur fixe

On comble chaque case vide avec une valeur choisie à l'avance (ex : 'inconnu', 0...), sans calcul particulier. Fonction utilisée : `fillna()`.

4. Remplacer par une valeur calculée

On comble les trous d'une colonne numérique avec une valeur représentative de la colonne : la médiane, préférée à la moyenne car elle n'est pas faussée par des valeurs extrêmes. Fonctions utilisées : `fillna()` avec `mean()` ou `median()`.



EXERCICE

Traiter les valeurs manquantes

1. Sur le même dataset, comptez les valeurs manquantes par colonne avec `isna().sum()`.

```
for c in ['Item', 'Payment Method', 'Location']:
    df[c] = df[c].replace('UNKNOWN', np.nan)
df.isna().sum()
```

2. Pour la colonne `Item`, supprimez les lignes où elle est vide (non restructurable).

```
df = df.dropna(subset=['Item'])
```

3. Pour la colonne `Price Per Unit`, remplacez les valeurs manquantes par la médiane.

```
df['Price Per Unit'] = pd.to_numeric(df['Price Per Unit'], errors='coerce')
df['Price Per Unit'] = df['Price Per Unit'].fillna(df['Price Per Unit'].median())
```

4. Pour la colonne `Payment Method`, remplacez les valeurs manquantes par 'inconnu'.

```
df['Payment Method'] = df['Payment Method'].fillna('inconnu')
```

5. Vérifiez qu'il ne reste plus aucune valeur manquante avec `isna().sum()`.

```
df.isna().sum()
```

Données aberrantes

- **outlier** valeur extrême, très éloignée du reste des données
- **boxplot** repère visuellement les valeurs hors norme
- $IQR = Q3 - Q1$ écart interquartile, mesure la dispersion centrale
- **bornes** = $Q1 - 1.5 \cdot IQR$ / $Q3 + 1.5 \cdot IQR$ au-delà : valeur aberrante

COMMENT ÇA MARCHE

1. Visualiser la distribution

On affiche un graphique qui résume la répartition des valeurs d'une colonne et fait ressortir visuellement les points isolés, très éloignés du reste. Outil utilisé : le boxplot (boîte à moustaches).

2. Mesurer la dispersion normale

On calcule l'écart entre le premier quart et le dernier quart des valeurs (l'IQR), qui représente la dispersion "habituelle" des données, sans être influencé par les valeurs extrêmes. Fonction utilisée : `quantile()`.

3. Définir des bornes et isoler les valeurs extrêmes

On calcule une borne basse et une borne haute autour de la dispersion normale (règle des $1,5 \times IQR$) ; tout point en dehors de cet intervalle est considéré comme une valeur aberrante à examiner.



EXERCICE

Détecter les valeurs aberrantes

1. Sur la colonne Price Per Unit, affichez un boxplot avec Matplotlib.

```
import matplotlib.pyplot as plt
plt.boxplot(df['Price Per Unit']); plt.title('Price Per Unit'); plt.show()
```

2. Calculez Q1, Q3 et l'IQR avec quantile().

```
Q1 = df['Price Per Unit'].quantile(0.25)
Q3 = df['Price Per Unit'].quantile(0.75)
IQR = Q3 - Q1
```

3. Déterminez les bornes et isolez les lignes aberrantes.

```
bb, bh = Q1 - 1.5*IQR, Q3 + 1.5*IQR
aberrants = df[(df['Price Per Unit']<bb) | (df['Price Per Unit']>bh)]
```

Encodage des variables

- **pourquoi** un modèle ne comprend que des nombres, pas du texte
- **Label Encoding** attribue un entier à chaque catégorie (ordre implicite)
- **One-Hot Encoding** crée une colonne binaire par catégorie (pas d'ordre)
- `pd.get_dummies()` applique le One-Hot Encoding en une ligne

COMMENT ÇA MARCHE

1. Comprendre le besoin

Un modèle de Machine Learning ne comprend que des nombres : toute colonne texte doit d'abord être transformée avant de pouvoir être utilisée dans un calcul ou un entraînement.

2. One-Hot Encoding

Pour une variable sans ordre naturel (ex : une catégorie de produit), on crée une colonne binaire (0/1) par valeur distincte, sans introduire de fausse hiérarchie entre les catégories. Fonction utilisée : `get_dummies()`.

3. Label Encoding

Pour une variable avec un vrai ordre logique (ex : petit < moyen < grand), on attribue simplement un entier croissant à chaque catégorie, en respectant cet ordre. Fonction utilisée : `map()` avec un dictionnaire d'ordre.



EXERCICE

Encoder les variables catégorielles

1. Identifiez les colonnes catégorielles (Item, Payment Method, Location).

```
df[['Item', 'Payment Method', 'Location']].nunique()
```

2. Appliquez un One-Hot Encoding sur 'Item'.

```
df_encode = pd.get_dummies(df, columns=['Item'])
```

3. Comparez le nombre de colonnes avant/après encodage.

```
print(df.shape[1], '->', df_encode.shape[1])
```

4. Appliquez un Label Encoding manuel sur une variable ordinale (ex : taille).

```
ordre = {'petit':0, 'moyen':1, 'grand':2}
df['taille_enc'] = df['taille'].map(ordre)
```

Normalisation

- **pourquoi** des échelles différentes faussent la contribution des variables
- **Min-Max** ramène les valeurs entre 0 et 1
- **Standardisation (z-score)** moyenne à 0, écart-type à 1
- **MinMaxScaler** / **StandardScaler** outils scikit-learn prêts à l'emploi

COMMENT ÇA MARCHE

1. Comprendre le besoin

Des variables numériques sur des échelles très différentes (ex : prix en dizaines vs âge en unités) faussent certains modèles, qui donnent alors trop d'importance à la variable avec les plus grandes valeurs.

2. Min-Max

On ramène toutes les valeurs d'une colonne entre 0 et 1, proportionnellement à son minimum et son maximum. Outil utilisé : MinMaxScaler.

3. Standardisation (z-score)

On recentre les valeurs autour d'une moyenne de 0 avec un écart-type de 1, sans les borner entre 0 et 1. Outil utilisé : StandardScaler.



EXERCICE

Normaliser les variables numériques

1. Appliquez un MinMaxScaler sur 'Price Per Unit' -> prix_norm.

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
df['prix_norm'] = MinMaxScaler().fit_transform(df[['Price Per Unit']])
```

2. Appliquez un StandardScaler sur 'Quantity' -> quantite_std.

```
df['quantite_std'] = StandardScaler().fit_transform(df[['Quantity']])
```

3. Affichez describe() avant/après pour vérifier les nouvelles échelles.

```
df[['prix_norm', 'quantite_std']].describe()
```

Analyse exploratoire (EDA)

- `df.describe()` statistiques clés de chaque colonne numérique
- `df[c].value_counts()` compte les occurrences par catégorie
- `df.corr()` matrice de corrélation entre variables numériques
- `df.groupby(c).mean()` moyenne par groupe, pour croiser des variables

COMMENT ÇA MARCHE

1. Vue d'ensemble statistique

On obtient en une seule commande les statistiques clés (moyenne, écart-type, min, max, quartiles) de toutes les colonnes numériques. Fonction utilisée : `describe()`.

2. Répartition d'une variable catégorielle

On compte combien de fois chaque catégorie apparaît, pour identifier la plus fréquente. Fonction utilisée : `value_counts()`.

3. Corrélations entre variables numériques

On mesure, pour chaque paire de colonnes numériques, si elles varient ensemble ou indépendamment l'une de l'autre. Fonction utilisée : `corr()`.

4. Moyenne par groupe

On regroupe les lignes par catégorie puis on calcule une statistique (ex : moyenne) pour chaque groupe, afin de croiser une variable catégorielle avec une variable numérique. Fonction utilisée : `groupby()`.



EXERCICE

Explorer le dataset nettoyé

1. Affichez `describe()` et trouvez la colonne au plus grand écart-type.

```
df.describe().loc['std'].idxmax()
```

2. Affichez `value_counts()` sur 'Item' : quel est le produit le plus fréquent ?

```
df['Item'].value_counts().idxmax()
```

3. Calculez la matrice de corrélation avec `corr()`.

```
df.corr(numeric_only=True)
```

4. Avec `groupby()`, calculez le prix moyen par produit et le plus cher.

```
m = df.groupby('Item')['Price Per Unit'].mean().sort_values(ascending=False)  
m.idxmax()
```

Matplotlib

- `plt.plot(x, y)` courbe / évolution
- `plt.bar(x, y)` diagramme en barres, comparaisons
- `plt.hist(data)` histogramme, distribution des valeurs
- `plt.scatter(x, y)` nuage de points, relation entre 2 variables
- `plt.title()` / `.xlabel()` toujours nommer titre et axes

EN PRATIQUE

```
import matplotlib.pyplot as plt

plt.bar(df['categorie'],
        df.groupby('categorie')['prix'].mean())
plt.title('Prix moyen par categorie')
plt.xlabel('Categorie')
plt.ylabel('Prix moyen (EUR)')
plt.show()
```



EXERCICE

Visualiser avec Matplotlib

1. Créez un histogramme de 'Price Per Unit' pour observer sa distribution.

```
plt.hist(df['Price Per Unit'], bins=15)
plt.title('Distribution des prix'); plt.xlabel('Prix'); plt.ylabel('Fréquence'); plt.show()
```

2. Créez un diagramme en barres du prix moyen par produit, titre et axes inclus.

```
m = df.groupby('Item')['Price Per Unit'].mean()
plt.bar(m.index, m.values); plt.title('Prix moyen par produit'); plt.show()
```

3. Créez un nuage de points Quantity / Price Per Unit.

```
plt.scatter(df['Quantity'], df['Price Per Unit'])
plt.title('Prix selon la quantité'); plt.show()
```

4. Vérifiez que chaque graphique a un titre clair et des axes nommés.

```
# plt.title(...) / plt.xlabel(...) / plt.ylabel(...) sur chaque figure
```

- **interactif** zoom, survol, filtrage directement dans le graphique
- **px.bar()** / **px.histogram()** mêmes graphiques que Matplotlib, interactifs
- **px.scatter()** nuage de points avec info-bulles au survol
- **fig.show()** ouvre le graphique interactif

```
import plotly.express as px

fig = px.scatter(df, x='age', y='prix',
                 color='categorie',
                 title='Prix selon l age')

fig.show()
```



EXERCICE

Visualiser avec Plotly

1. Nuage de points interactif Quantity / Price Per Unit coloré par Item.

```
import plotly.express as px  
px.scatter(df, x='Quantity', y='Price Per Unit', color='Item').show()
```

2. Histogramme interactif de 'Price Per Unit'.

```
px.histogram(df, x='Price Per Unit', nbins=15).show()
```

Matplotlib ou Plotly ?

Matplotlib

- Statique, léger, rapide à générer
- Standard pour rapports et publications
- Contrôle fin de chaque détail visuel
- Idéal pour figures fixes, imprimées

Plotly

- Interactif : zoom, survol, filtrage
- Idéal pour l'exploration de données
- Bien adapté au web / dashboards
- Plus lourd pour un simple export image